



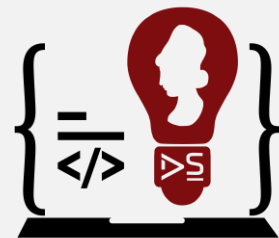
INGENIERÍA EN DESARROLLO DE SOFTWARE

CICLO 1/2025 – ASIGNATURA:

MANEJO DE ESTRUCTURA DE DATOS

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura



Ingeniería en Desarrollo
de Software



TEMA:

MANEJO DE ESTRUCTURA DE DATOS

03



MANEJO DE ESTRUCTURA DE DATOS

Las estructuras de datos en programación son un modo de representar información en una computadora, aunque, además, cuentan con un comportamiento interno. ¿Qué significa? Que se rige por determinadas reglas/restricciones.



MANEJO DE ESTRUCTURA DE DATOS

¿Para qué sirven las estructuras de datos?

En el ámbito de la informática, las estructuras de datos son aquellas que nos permiten organizar la información de manera eficiente, y en definitiva diseñar la solución correcta para un determinado problema.



MANEJO DE ESTRUCTURA DE DATOS

¿Cuáles son los tipos de estructuras de datos?

Primero, debemos diferenciar entre estructura de dato estática y estructura de dato dinámica.



MANEJO DE ESTRUCTURA DE DATOS

Las estructuras de datos estáticas son aquellas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa.



MANEJO DE ESTRUCTURA DE DATOS

Una **estructura de datos dinámica** es aquella en la que el tamaño ocupado en memoria puede modificarse durante la ejecución del programa.



MANEJO DE ESTRUCTURA DE DATOS

Las **estructuras de datos lineales** son aquellas en las que los elementos ocupan lugares sucesivos en la estructura y cada uno de ellos tiene un único sucesor y un único predecesor.



MANEJO DE ESTRUCTURA DE DATOS

Hay tres tipos de estructuras de datos lineales:

- Listas enlazadas
- Pilas
- Colas



PILAS

Para introducir esta estructura, recordemos la forma en que se apilan los platos en los restaurantes: una pila de platos se soporta sobre un muelle, cuando se retira un plato, los demás suben.



PILAS

Una pila (stack) es una estructura lineal a cuyos datos sólo se puede acceder por un solo extremo, denominado tope o cima (top).

En esta estructura sólo se pueden efectuar dos operaciones: añadir y eliminar un elemento.



PILAS

Si se meten varios elementos en la pila y después se sacan de ésta, el último elemento en entrar será el primero en salir.

Por esta razón se dice que la pila es una estructura LIFO (last in first out).



PILAS

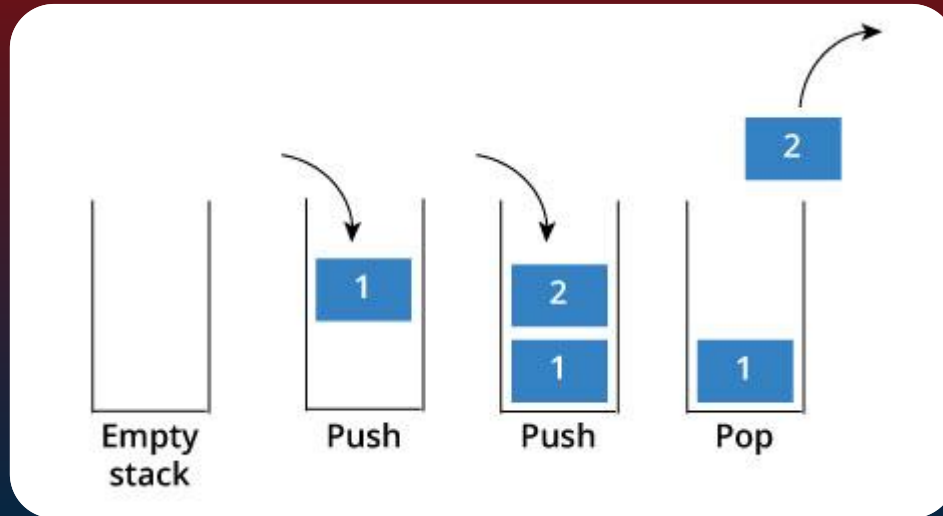
Una pila es una estructura de datos homogénea (elementos del mismo tipo), secuencial y de tamaño variable.

Sólo es posible un modo de acceso a esta estructura: a través de la cabeza de la pila.



PILAS

Esquema de una pila:





PILAS

La clase stack de java:

```
Stack pila= new Stack();  
pila.push("Hola");
```



PILAS

Para hacer uso de esta clase es necesario incluir al inicio del programa la siguiente línea:

```
import tools.*;
```

Constructor:

`stack()` Crea una pila inicialmente vacía.



PILAS

- `void push(int val)`: inserta el entero `val` en la pila.
- `boolean isEmpty()`: entrega verdadero si la pila esta vacía, falso en caso contrario.
- `int popInt()`: recupera el valor entero que se encuentra en el tope de la pila y lo elimina de ésta. Se produce un error en tiempo de ejecución si el elemento de la pila no es de tipo `int`.



COLAS

La cola se le considera un primo cercano de la pila.

La cola puede definirse como un contenedor de datos que funciona de acuerdo al principio FIFO(First Input First Output).



COLAS

En una cola los datos entran por un extremo llamado final (rear) y se insertan por el otro extremo llamado frente(front).

Una buena analogía de esta estructura de datos es un grupo de personas esperando en línea para entrar al cine.



COLAS

Hay varias formas de implementar una cola en la memoria de un ordenador, bien con vectores, bien en listas enlazadas.

En cualquier caso se necesitan dos variables que representen a los punteros FRENTE (f = front) y FINAL (r = rear).



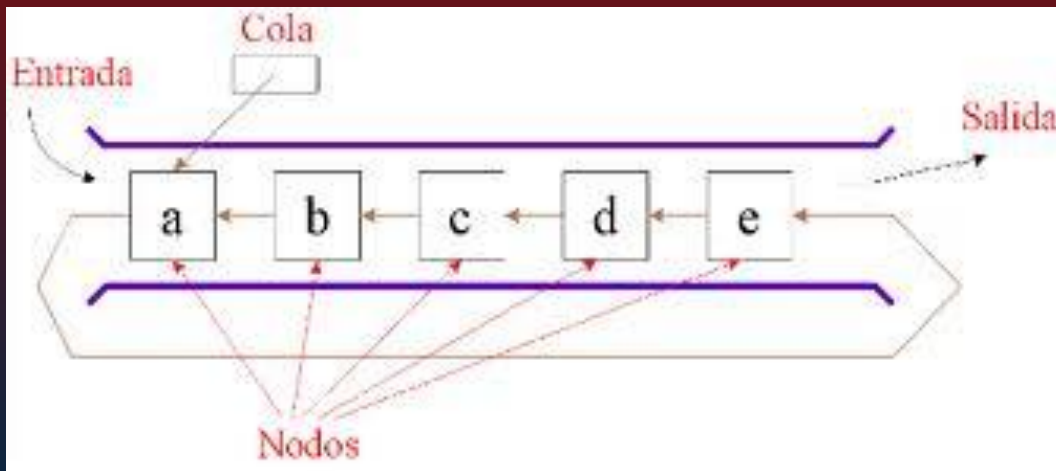
COLAS

El estado de COLA VACIA se manifiesta cuando f y r son ambas nulos en la implementación dinámica.
Facilitan la interconexión y el almacenamiento de datos en tránsito tanto en redes de ordenadores.



COLAS

Representación gráfica de una cola:





COLAS

Operaciones Básicas

- . Insertar. Agregar un elemento al final de la cola.
- . Remover. Remover el primer elemento de la cola.

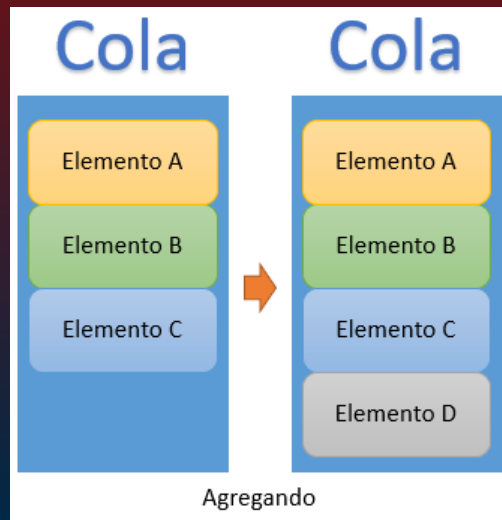
Operaciones Auxiliares

- . Llena: Verdadero cuando la cola esta llena.
- . Vacía: Verdadero cuando la cola esta vacía.



COLAS

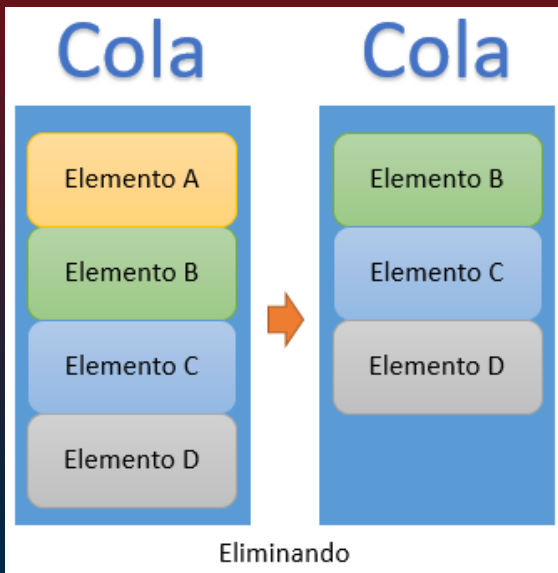
Operaciones Básicas: Agregando





COLAS

Operaciones Básicas: Eliminando





COLAS

Queues en Java

Admite las siguientes operaciones principales:

- Enqueue: Inserta un elemento al final de la queue.
- Dequeue: Elimina el objeto del principio de la queue y lo devuelve, reduciendo así su tamaño.



COLAS

- Peek: Devuelve el objeto al principio de la queue sin eliminarlo.
- IsEmpty: Verifica si la queue está vacía o no.
- Size: Devuelve el número total de elementos presentes en la queue.



COLAS

Ejemplo de Colas en JAVA:

```
1  import java.util.LinkedList;
2  import java.util.Queue;
3
4  class Main
5  {
6      public static void main(String[] args)
7      {
8          Queue<String> queue = new LinkedList<String>();
9
10         queue.add("A");    // Inserta `A` en la queue
11         queue.add("B");    // Inserta `B` en la queue
12         queue.add("C");    // Inserta `C` en la queue
13         queue.add("D");    // Inserta `D` en la queue
14
15         // Imprime el frente de la queue (`A`)
16         System.out.println("The front element is " + queue.peek());
17
18         queue.remove();    // eliminando el elemento frontal (`A`)
19         queue.remove();    // eliminando el elemento frontal (`B`)
20     }
```



COLAS

```
20
21     // Imprime el frente de la queue (`C`)
22     System.out.println("The front element is " + queue.peek());
23
24     // Devuelve el número total de elementos presentes en la queue
25     System.out.println("The queue size is " + queue.size());
26
27     // comprobar si la queue está vacía
28     if (queue.isEmpty()) {
29         System.out.println("The queue is empty");
30     }
31     else {
32         System.out.println("The queue is not empty");
33     }
34 }
35 }
```



Alguna pregunta?

GRACIAS!

